DP and Greedy

2026-1학기 부경대학교 알고리즘 스터디

목차

- DP
  - DP란?

    Overlapping Subproblems(겹치는 부분 문제),

    Optimal Substructure(최적 부분 구조)
  - 구현 방법

    탑 다운 방식(재귀), 바텀 업 방식(반복문)
  - 사용 예

    누적 합, 냅색 문제, LIS, 외판원 순회(비트 마스크)
  - 역추적
- Greedy
  - Greedy란?
  - DP와 비교
  - 최적해 증명 방법

    Greedy stays ahead, Certificate argument,

    Exchange argument

DP - DP란?

- DP(dynamic programming, 동적계획법)란?
  - 큰 문제를 작은 부분 문제로 분할해서 해결하고 작은 부분 문제의 정답을 저장(메모이제이션)해두어 푸는 방법
  - Dp를 사용하기 위해선 dp table과 점화식 정의를 해야 함
  - 시간복잡도를 예측해야 함

    팁: (dp의 시간복잡도) = (dp table의 크기) * (함수의 시간복잡도)

DP - dp 사용을 위한 조건

- Overlapping Subproblems(겹치는 부분 문제)
  - 같은 부분 문제를 2번 이상 풀어야 함
  - 만약 모든 부분 문제를 2번 이상 풀지 않는다면 dp를 쓰는 이유가 없음
- Optimal Substructure(최적 부분 구조)
  - 상위 부분 문제의 최적해를 하위 부분 문제의 최적해로 구성할 수 있을 때

DP - 구현 방법

- 바텀 업 방식(반복문)
- 탑 다운 방식(재귀)

- 예제로 살펴보자

피보나치 수 2 BOJ 2748

- 문제
  - n번째 피보나치 수를 출력하라.
  - n <= 90

피보나치 수 2 BOJ 2748

- 문제
  - n번째 피보나치 수를 출력하라.
  - n ≤ 90

- 풀이
  - n이 크기 때문에 재귀로 풀면 2^n 이므로 시간초과가 난다.
    →dp를 사용하면 O(n)로 풀이 가능
  - 정의: dp[n] = n번째 피보나치 수
  - 점화식: dp[n] = dp[n – 1] + dp[n – 2]

피보나치 수 2 BOJ 2748

- 바텀 업 방식(반복문)

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL), cout.tie(NULL);

    ll n;
    ll dp[91] = { 0, 1, };

    cin >> n;
    for (int i = 2; i ≤ n; i++)
        dp[i] = dp[i - 1] + dp[i - 2];

    cout << dp[n];

    return 0;
}
```

- 탑 다운 방식(재귀)

```cpp
#include <bits/stdc++.h>
using namespace std;

using ll = long long;

ll dp[91];

ll dfs(int n) {
    ll& ret = dp[n];
    if (ret ≠ -1) return ret;

    if (n < 2) return ret = n;
    else return ret = dfs(n - 1) + dfs(n - 2);
}

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL), cout.tie(NULL);

    int n;
    cin >> n;
    fill(&dp[0], &dp[91], -1);

    cout << dfs(n);

    return 0;
}
```

DP – 사용 예

- 누적 합
  - 수열에서 a번부터 b번까지의 구간 합을 구하려면 O(n)이 걸림

    → 더 개선할 수 있을까?

DP – 사용 예

- 누적 합
  - 수열에서 a번부터 b번까지의 구간 합을 구하려면 O(n)이 걸림
    → 더 개선할 수 있을까?
  - 정의: S[x] = 0번에서 x번까지의 합
  - 점화식: S[x] = S[x-1] + arr[x]
  - 0번부터 b번까지의 합(S[b])을 미리 계산 후 a번부터 b번까지의 합을 구할 때
    S[b] – S[a-1]로 구할 수 있음 -> O(1)

DP – 사용 예

- 누적 합
  - 2차원에서 구간 합을 구하려면 O(n^2)이 걸림

    → 1차원처럼 개선할 수 있을까?

  - 가능함, 여기선 다루지 않음

구간 합 구하기 4 BOJ 11659

- 문제
  - 수 n개가 주어질 때 i번째부터 j번째까지의 합을 구하는 질의가 m개 주어진다. 질의에 대한 답을 출력하라.
  - $1 \le n, m \le 100{,}000$, $1 \le i \le j \le n$

구간 합 구하기 4 BOJ 11659

- 문제
  - 수 n개가 주어지고 i번째부터 j번째까지의 합을 구하는 질의 m개가 주어진다. 질의에 대한 답을 출력하라.
  - $1 \le n, m \le 100{,}000$, $1 \le i \le j \le n$

- 풀이
  - 질의가 주어질 때마다 구간합을 구하면 nm의 시간이 걸림
    → dp를 사용하면 O(n+m)에 가능
  - 누적 합을 구한 뒤 질의가 들어올 때 S[j]-S[i-1]로 구할 수 있음

구간 합 구하기 4 BOJ 11659

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL), cout.tie(NULL);

    int n, m;
    vector<int> s, arr;

    cin >> n >> m;
    arr.resize(n + 1); s.resize(n + 1);
    for (int i = 1; i ≤ n; i++) {
        cin >> arr[i];
        s[i] = s[i - 1] + arr[i];
    }

    while (m--) {
        int i, j;
        cin >> i >> j;

        cout << s[j] - s[i-1] << "\n";
    }

    return 0;
}
```

DP – 사용 예

- 냅색 문제
  - W의 용량을 가진 가방이 있고 w의 비용과 v만큼의 가치를 가지는 아이템들이 n개 있을 때 가방에 담은 아이템의 최대 가치 합을 구하는 문제
  - 각각의 아이템을 넣거나 넣지 않는 경우를 고려해보자.

    n번째를 넣는 경우 n-1개의 아이템이 있고 용량이 W-arr[n].w인 가방이 있을 때

    n번째를 넣지 않는 경우 n-1개의 아이템이 있고 용량이 W인 가방이 있을 때

    → 둘 중 더 큰 값을 선택하면 최대 가치합을 구할 수 있음

  - 정의: dp[i][w] = i까지의 아이템들을 고려했을 때 용량이 w인 가방에 넣을 수 있는 아이템의 최대 가치 합
  - 점화식: dp[i][w] = max(dp[i-1][w], dp[i-1][w-arr[i].w]+arr[i].v)
  - 시간복잡도 O(nW)

평범한 배낭 BOJ 12865

- 문제
  - 무게가 w이고 가치가 v인 물품이 n개 있고 가방에 최대 k만큼의 무게를 넣을 수 있는 배낭이 있다. 이 때 배낭에 넣을 수 있는 물품들의 가치의 최댓값을 출력하라.
  - $1 \le n \le 100$, $1 \le k \le 100{,}000$, $1 \le w \le 100{,}000$, $0 \le v \le 1{,}000$

평범한 배낭 BOJ 12865

- 문제
  - 무게가 w이고 가치가 v인 물품이 n개 있고 가방에 최대 k만큼의 무게를 넣을 수 있는 배낭이 있다. 이 때 배낭에 넣을 수 있는 물품들의 가치의 최댓값을 출력하라.
  - $1 \le n \le 100$, $1 \le k \le 100{,}000$, $1 \le w \le 100{,}000$, $0 \le v \le 1{,}000$

- 풀이
  - 냅색 문제임을 알 수 있음
  - 시간복잡도는 $O(nk)$이므로 시간 안에 동작함

평범한 배낭 BOJ 12865

```cpp
#include <bits/stdc++.h>
using namespace std;

using pii = pair<int, int>;

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL), cout.tie(NULL);

    int n, k;
    vector<pii> items;
    int dp[101][100'001] = { 0, };

    cin >> n >> k;
    items.resize(n + 1);
    for (int i = 1; i ≤ n; i++) {
        cin >> items[i].first >> items[i].second;
    }

    for (int i = 1; i ≤ n; i++) {
        for (int j = 1; j ≤ k; j++) {
            if (j - items[i].first < 0) dp[i][j] = dp[i - 1][j];
            else dp[i][j] = max(dp[i - 1][j], dp[i - 1][j - items[i].first] + items[i].second);
        }
    }

    cout << dp[n][k];

    return 0;
}
```

DP 사용 예

- **LIS**
  - 가장 긴 증가 부분 수열을 찾는 문제
  - j번째까지 봤을 때 k이고 arr[j]보다 arr[i]가 크다면 i번째까지 봤을 때 k+1이 되는 것을 알 수 있다. 이 것을 모든 j < i에 대하여 해보고 최대인 것을 고르면 된다.
  - 정의:LIS[i] = i번까지 봤을 때 최장 증가 수열의 길이
  - 점화식: $LIS[i] = \max_{j<i}\left(LIS[j] + (arr[i] > arr[j])\right)$
  - 시간복잡도: O(n^2)

가장 긴 증가하는 부분 수열 BOJ 11053

- 문제
  - 길이가 n인 수열이 주어질 때 가장 긴 증가하는 부분 수열의 길이를 출력하라.
  - 1 ≤ n ≤ 1,000, 1 ≤ a_i ≤ 1,000

가장 긴 증가하는 부분 수열 BOJ 11053

- 문제
  - 길이가 n인 수열이 주어질 때 가장 긴 증가하는 부분 수열의 길이를 출력하라.
  - 1 ≤ n ≤ 1,000, 1 ≤ a_i ≤ 1,000

- 풀이
  - LIS를 찾으면 됨
  - O(n^2)이니 시간 안에 동작함

가장 긴 증가하는 부분 수열 BOJ 11053

```cpp
#include <bits/stdc++.h>
using namespace std;

int dp[1001];
vector<int> arr;

int dfs(int d) {
    int& ret = dp[d];
    if (ret != -1) return ret;

    if (d == 0) return ret = 1;

    ret = 1;
    for (int i = 0; i < d; i++) {
        if (arr[d] > arr[i])
            ret = max(ret, dfs(i) + 1);
    }

    return ret;
}
```

```cpp
int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL), cout.tie(NULL);

    int n;
    cin >> n;
    arr.resize(n);
    for (int i = 0; i < n; i++) {
        cin >> arr[i];
    }
    fill(&dp[0], &dp[1000], -1);

    int res = 0;
    for (int i = 0; i < n; i++) {
        res = max(res, dfs(i));
    }
    cout << res;

    return 0;
}
```

DP 사용 예

- **외판원 순회(비트마스크)**
  - 임의의 정점에서 시작해 모든 정점을 한 번만 방문하면서 순회하고 마지막에 시작한 정점으로 돌아올 때 최소 비용의 이동 순서를 구하는 문제
  - 현재 방문한 정점들의 집합이 s이고 현재 위치한 정점을 v라고 하면 최적 부분 구조가 된다.
  - 정의: TSP[s][v] = 이미 방문한 정점들의 집합이 s이면서 현재 위치한 정점이 v일 때 남은 방문하지 않은 정점을 방문하고 시작 지점으로 돌아올 때 최소 비용
    - ※ 집합 s는 비트마스크로 구현, 이진수로 1101이면 3, 2, 0번 이 집합에 포함되어 있음을 의미함
  - 점화식: $\mathrm{TSP[s][v]} = \min_{\mathrm{nxt} \in \mathrm{adj(v)}} \left(\mathrm{TSP[s][nxt]} + (\mathrm{adj[v][nxt]})\right)$
    - ※ adj(v): v에 인접한 정점들의 집합
    - ※ adj[v][nxt]: v에서 nxt로 가는 간선의 가중치
  - 시간복잡도: O($n^2 2^n$)

외판원 순회 BOJ 2098

- 문제
  - 도시의 수 n과 각 도시 간의 이동 비용이 w[i][j](i에서 j로 가기위한 비용)로 주어진다. w[i][j]가 0이면 길이 없다. 이 때 외판원 순회에 필요한 최소 비용을 출력하라.
  - $2 \le n \le 16$, $w[i][j] \le 1{,}000{,}000$

외판원 순회 BOJ 2098

- 문제
  - 도시의 수 n과 각 도시 간의 이동 비용이 w[i][j](i에서 j로 가기위한 비용)로 주어진다. w[i][j]가 0이면 길이 없다. 이 때 외판원 순회에 필요한 최소 비용을 출력하라.
  - 2 ≤ n ≤ 16, w[i][j] ≤ 1,000,000

- 풀이
  - 외판원 순회를 하는데 필요한 최소 비용을 찾으면 됨
  - dp를 사용하면 외판원 순회에 O($n^2 2^n$)가 걸리니 시간 안에 작동함

외판원 순회 BOJ 2098

```cpp
#include <bits/stdc++.h>
using namespace std;

using ll = long long;

int n;
ll dp[(1<<16)][16];
ll adj[16][16];

ll dfs(int s, int v) {
    ll& ret = dp[s][v];
    if (ret != -1) return ret;

    ret = numeric_limits<int>::max();
    if (s == (1 << n) - 1) {
        if (adj[v][0] == 0) return ret;
        else return ret = adj[v][0];
    }

    for (int nxt = 0; nxt < n; nxt++) {
        if ((s & (1 << nxt)) == 0 && adj[v][nxt])
            ret = min(ret, dfs(s | (1 << nxt), nxt) + adj[v][nxt]);
    }

    return ret;
}
```

```cpp
int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL), cout.tie(NULL);

    cin >> n;
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            cin >> adj[i][j];
        }
    }
    fill(&dp[0][0], &dp[(1<<16)-1][16], -1);

    // 0번 정점에서 시작
    cout << dfs(1, 0);

    return 0;
}
```

DP - 역추적

- 역추적을 하는 이유
  - 현재 해를 구하기까지의 경로를 추적하고 싶기 때문
  - ex) LIS의 경우 실제 최대 증가 부분 수열을 찾을 수 있음
- 역추적 방법
  - dp를 돌릴 때 현재 해를 구성하는 부분 문제를 저장
  - dp를 다 돌린 후 dp table을 보고 부분 문제가 무엇인지 찾아서 해를 저장
    - ※ 최적 부분 구조를 가지기 때문에 가능
  - 예제를 통해 살펴보자.

가장 긴 증가하는 부분 수열 4 BOJ 14002

- 문제
  - 길이 n의 수열이 주어질 때 가장 긴 증가하는 부분 수열의 길이와 구한 부분 수열을 출력하라.
  - 1 ≤ n ≤ 1,000, 1 ≤ a_i ≤ 1,000

- 풀이
  - 앞에서 본 가장 긴 증가하는 부분 수열 문제에서 구한 부분 수열을 출력하는 부분이 추가된 문제
  - dp를 사용해서 길이를 구하고 역추적을 해서 실제 부분 수열을 구함

가장 긴 증가하는 부분 수열 4 BOJ 14002

- LIS 역추적
  - parent 배열을 사용해 역추적

    ※ dfs 함수에서 parent를 채움

```cpp
#include <bits/stdc++.h>
using namespace std;

int dp[1001];
vector<int> arr, parent;

int dfs(int d) {
    int& ret = dp[d];
    if (ret != -1) return ret;

    if (d == 0) return ret = 1;

    ret = 1;
    for (int i = 0; i < d; i++) {
        if (arr[d] > arr[i] && ret < dfs(i) + 1) {
            ret = dfs(i) + 1;
            parent[d] = i;
        }
    }

    return ret;
}
```

```cpp
int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL), cout.tie(NULL);

    int n;
    cin >> n;
    arr.resize(n); parent.resize(n);
    for (int i = 0; i < n; i++) {
        cin >> arr[i];
    }
    fill(&dp[0], &dp[1000], -1);
    fill(&parent[0], &parent[n], -1);

    int res = 0;
    for (int i = 0; i < n; i++) {
        res = max(res, dfs(i));
    }
    cout << res << "\n";

    // 역추적
    vector<int> lis;
    int i;
    for (i = n - 1; i >= 0; i--) {
        if (res == dp[i]) break;
    }
    while (i != -1) {
        lis.push_back(arr[i]);
        i = parent[i];
    }

    reverse(lis.begin(), lis.end());
    for (int x : lis) {
        cout << x << " ";
    }

    return 0;
}
```

가장 긴 증가하는 부분 수열 4 BOJ 14002

- LIS 역추적
  - dp table을 사용해 직접적으로 역추적

```cpp
#include <bits/stdc++.h>
using namespace std;

int dp[1001];
vector<int> arr;

int dfs(int d) {
    int& ret = dp[d];
    if (ret != -1) return ret;

    if (d == 0) return ret = 1;

    ret = 1;
    for (int i = 0; i < d; i++) {
        if (arr[d] > arr[i])
            ret = max(ret, dfs(i) + 1);
    }

    return ret;
}
```

```cpp
int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL), cout.tie(NULL);

    int n;
    cin >> n;
    arr.resize(n);
    for (int i = 0; i < n; i++) {
        cin >> arr[i];
    }
    fill(&dp[0], &dp[1000], -1);

    int res = 0;
    for (int i = 0; i < n; i++) {
        res = max(res, dfs(i));
    }
    cout << res << "\n";

    // 역추적
    vector<int> lis;
    int t = res;
    for (int i = n - 1; i >= 0; i--) {
        if (t == dp[i]) lis.push_back(arr[i]), t--;
    }

    reverse(lis.begin(), lis.end());
    for (int x : lis) {
        cout << x << " ";
    }

    return 0;
}
```

Greedy – Greedy란?

- Greedy란?
  - 각 단계에서 국소적으로 최적의 선택을 하는 휴리스틱 알고리즘
    - ※ 휴리스틱: 정확한 솔루션을 근사화
    - ※ 국소적: 전체 중 특정한 어느 한 부분이나 좁은 영역에만 한정되어 영향을 미치는 상태
  - 다만 일부(ps에선 그리디를 사용하는 대부분의 문제)는 그리디로 최적의 해를 얻을 수 있다.

Greedy - DP와 비교

- DP와 비교
  - DP와 동일하게 최적 부분 구조를 가짐
  - DP와는 다르게 모든 케이스를 체크하지 않음
  - DP는 무조건 정답을 얻을 수 있는 반면 그리디는 문제에 따라 정답이 나오지 않을 수 있음

Greedy – 최적해 증명 방법

- Greedy stays ahead
  - 그리디한 방법이 항상 가상의 최적해보다 더 좋은 선택을 한다는 것을 증명하는 방법
- Certificate argument
  - 찾아낸 답이 최적해라는 증거를 제출하는 방법
- Exchange argument
  - 최적해에서 조금씩 수정하여 찾아낸 답이 최적해보다 나쁘지 않다는 것을 보이는 방법

  - 블로그를 참고함, 설명이 잘 되어 있으니 궁금한 사람은 보는 것을 추천

회의실 배정 BOJ 1931

- 문제
  - n개의 회의가 있고 각 회의에 대하여 시작 시간과 끝나는 시간이 주어진다. 각 회의는 겹칠 수 없으며 한 회의가 끝나는 것과 동시에 다른 회의가 시작하는 것은 가능하다. 이 때 최대 회의 개수를 출력하라.
  - 1 ≤ n ≤ 100,000, 시작 시간과 끝나는 시간은 $2^{31} - 1$보다 작거나 같다.

회의실 배정 BOJ 1931

- 문제
  - n개의 회의가 있고 각 회의에 대하여 시작 시간과 끝나는 시간이 주어진다. 각 회의는 겹칠 수 없으며 한 회의가 끝나는 것과 동시에 다른 회의가 시작하는 것은 가능하다. 시작 시간과 끝나는 시간이 같을 수 있다. 이 때 최대 회의 개수를 출력하라.
  - 1 ≤ n ≤ 100,000, 시작 시간과 끝나는 시간은 $2^{31} - 1$보다 작거나 같다.
- 풀이
  - 끝나는 시간을 기준으로 오름차순 정렬을 해서 각 회의를 순서대로 보면서 할 수 있으면 하고 아니면 버림
  - 시작 시간과 끝나는 시간이 같은 경우 예외이기 때문에 정렬 시 끝나는 시간이 같다면 시작 시간을 기준으로 오름차순 정렬을 함

회의실 배정 BOJ 1931

- 증명
  - Greedy stays ahead(최적해보다 그리디가 항상 좋음)로 증명 가능
  - 가상의 최적해(O)가 있다고 가정할 때 각 회의가 한 쌍씩 매칭된다면 그리디 방식(G)이 가상의 최적해보다 항상 좋다고 말할 수 있음
  - G의 i번째 회의의 시작 시간 b(G_i), 끝나는 시간 e(G_i), O의 i'번째 회의의 시작 시간 b(O_i), 끝나는 시간 e(O_i)
  - b(G_i) ≤ e(G_i), b(O_i) ≤ e(O_i)

회의실 배정 BOJ 1931

- 증명
  - 모든 i에 대하여 $e(G_i) \le e(O_i)$임을 보이면 됨
  - 수학적 귀납법으로 증명 가능
  - $e(G_1) \le e(O_1)$는 그리디 방식은 가장 끝나는 시간이 빠른 것을 처음 선택하니 자명함
  - $e(G_{i-1}) \le e(O_{i-1})$이라 가정할 때 G_i가 없는 경우는 $e(G_{i-1}) \le e(O_{i-1}) \le f(O_i)$이기 때문에 G_i로 O_i를 선택하면 되니 모순이 됨
  - 이제 $e(G_i) \le e(O_i)$인데 귀류법으로 $e(G_i) > e(O_i)$는 그리디 방법은 항상 할 수 있는 회의 중 끝나는 시간이 가장 빠른 회의를 고르기 때문에 모순이 되니 따라서 $e(G_i) \le e(O_i)$이다

회의실 배정 BOJ 1931

```cpp
#include <bits/stdc++.h>
using namespace std;

using pii = pair<int, int>;

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL), cout.tie(NULL);

    int n;
    vector<pii> arr;

    cin >> n;
    arr.resize(n);
    for (int i = 0; i < n; i++) {
        cin >> arr[i].first >> arr[i].second;
    }

    sort(arr.begin(), arr.end(), [](const pii& a, const pii& b) {
        if (a.second == b.second) {
            return a.first < b.first;
        }
        else {
            return a.second < b.second;
        }
    });

    int res = 0, end = -1;
    for (int i = 0; i < n; i++) {
        if (end <= arr[i].first) {
            res++;
            end = arr[i].second;
        }
    }

    cout << res;

    return 0;
}
```

강의실 배정 BOJ 11000

- 문제
  - n개의 강의가 있고 각 강의에 대하여 시작 시간과 끝나는 시간이 주어진다. 각 강의는 겹칠 수 없으며 한 강의가 끝나는 것과 동시에 다른 강의가 시작하는 것은 가능하다. 써야 하는 강의실의 최소 개수를 출력하라.
  - 1 ≤ n ≤ 200,000, 각 시간은 10^9 이하다.

강의실 배정 BOJ 11000

- 문제
  - n개의 강의가 있고 각 강의에 대하여 시작 시간과 끝나는 시간이 주어진다. 각 강의는 겹칠 수 없으며 한 강의가 끝나는 것과 동시에 다른 강의가 시작하는 것은 가능하다. 써야 하는 강의실의 최소 개수를 출력하라.
  - 1 ≤ n ≤ 200,000, 각 시간은 10^9 이하다.

- 풀이
  - 시간이 겹치면 새로운 강의실을 하나 사용하고 겹치지 않으면 기존의 강의실을 계속해서 사용하고 새로운 강의실을 볼 때마다 현재 시간에 쓰고 있는 강의실을 개수를 기록함

강의실 배정 BOJ 11000

- 증명
  - Certificate Argument(답이 최적해라는 증거 제출)로 증명 가능
  - 이 문제의 증명은 단순한데 강의실 최소 개수는 강의실을 가장 많이 쓰는 시간대에 쓰는 강의실의 개수와 같을 것이니 같은 시간대에 최소한의 강의실을 사용하도록 하면 답을 구할 수 있음

강의실 배정 BOJ 11000

```cpp
#include <bits/stdc++.h>
using namespace std;

using pii = pair<int, int>;

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL), cout.tie(NULL);

    int n;
    vector<pii> arr;

    cin >> n;
    for (int i = 0; i < n; i++) {
        int s, e;
        cin >> s >> e;

        arr.push_back(make_pair(s, 1));
        arr.push_back(make_pair(e, -1));
    }

    sort(arr.begin(), arr.end(), [](const pii& a, const pii& b) {
        if (a.first == b.first) return a.second < b.second;
        else return a.first < b.first;
        });

    int res = 0, mx = 0;
    for (pii& t : arr) {
        if (t.second == 1) {
            res++;
            mx = max(mx, res);
        }
        else {
            res--;
        }
    }

    cout << mx;

    return 0;
}
```

소가 길을 건너간 이유 4 BOJ 14464

- 문제
  - 닭은 c마리 존재하고 각 닭은 t_i초에만 소를 도울 수 있다. 소는 n마리 있으며 각 소는 a_i초에서 b_i초에만 길을 건널 수 있다. 소가 닭의 도움을 받으려면 $a_i \le t_i \le b_i$를 만족해야 한다. 이 때 도움을 받을 수 있는 소가 최대 몇 마리인지 출력하라.
  - 1 ≤ c, n ≤ 20,000, a, b, t는 1,000,000,000이하이고 같을 수 있다.

소가 길을 건너간 이유 4 BOJ 14464

- 문제
  - 닭은 c마리 존재하고 각 닭은 t_i초에만 소를 도울 수 있다. 소는 n마리 있으며 각 소는 a_i초에서 b_i초에만 길을 건널 수 있다. 소가 닭의 도움을 받으려면 $a_i \le t_i \le b_i$를 만족해야 한다. 이 때 도움을 받을 수 있는 소가 최대 몇 마리인지 출력하라.
  - 1 ≤ c, n ≤ 20,000, a, b, t는 1,000,000,000이하이고 같을 수 있다.

- 풀이
  - 닭을 오름차순으로 정렬하고 순서대로 소의 b_i가 가장 작은 값에 할당할 수 있으면 하고 아니면 넘김
  - 실제 구현에선 소를 오름차순 정렬하고 a_i가 t보다 작거나 같다면

소가 길을 건너간 이유 4 BOJ 14464

- 증명
  - Exchange Argument(그리디 방식에서 다른 해로 교체 시 이득이 아님)로 증명 가능
  - $\mathrm{t_i}$에 가장 작은 값이 아닌 $(\mathrm{b_j} < \mathrm{b_k})$를 선택한 경우를 생각해보자. 그런 경우에도 $t_{i+1} \leq \mathrm{b_j}$이어서 도움을 받을 수 있는 소의 마릿수가 같을 수도 있지만 아닌 경우 $\mathrm{b_j}$를 선택한 경우보다 마릿수가 더 작아짐

소가 길을 건너간 이유 4 BOJ 14464

```cpp
#include <bits/stdc++.h>
using namespace std;

using pii = pair<int, int>;

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL), cout.tie(NULL);

    int c, n;
    vector<int> chicken;
    vector<pii> cow;
    priority_queue<int, vector<int>, greater<int>> pq;

    cin >> c >> n;
    chicken.resize(c); cow.resize(n);
    for (int i = 0; i < c; i++) {
        cin >> chicken[i];
    }
    for (int i = 0; i < n; i++) {
        cin >> cow[i].first >> cow[i].second;
    }
    sort(chicken.begin(), chicken.end());
    sort(cow.begin(), cow.end());
```

```cpp
    int res = 0, idx = 0;
    for (int i = 0; i < c; i++) {
        while (idx < n && cow[idx].first ≤ chicken[i]) {
            pq.push(cow[idx].second);
            idx++;
        }

        while (pq.size() && pq.top() < chicken[i]) pq.pop();

        if (pq.size()) {
            res++;
            pq.pop();
        }
    }

    cout << res;

    return 0;
}
```

그 외 연습 문제

- 그리디인지 dp인지 파악하기, dp이면 점화식과 정의 생각해보기, 그리디이면 왜 그리디로 풀 수 있는지 증명 생각해보기
- 1149 RGB거리
- 13305 주유소
- 11047 동전 0
- 2294 동전 2